

Binarizzazione di una matrice

Progetto - Sistemi Concorrenti e Paralleli M

A.A. 2025-2026

Davide Chirichella - 0001222371

Contents



1	Proposta di progetto	4
1.a	Obiettivi	5
2	Implementazione Seriale	7
2.a	Allocazione delle matrici	8
2.b	Algoritmo	9
3	Implementazione Parallela: MPI	11
3.a	Distribuzione del carico di lavoro	12
3.b	Allocazione e scambio delle Ghost Rows	14
3.c	Calcolo elementi dell'intorno e sogliatura	19
3.d	Liberazione memoria allocata e raccolta risultati	21
4	Implementazione Parallela: OpenMP	22
4.a	Considerazioni generali	23
4.b	Scelte implementative	24
4.c	Algoritmo	26
5	Benchmark	27
5.a	Setup sperimentale	28

Contents (ii)



5.b MPI	30
5.b.a Strong Scaling intra-nodo	30
5.b.b Weak Scaling intra-nodo	32
5.b.c Strong Scaling inter-nodo	34
5.b.d Weak Scaling inter-nodo	36
5.c OpenMP	38
5.c.a Strong Scaling	38
5.c.b Weak Scaling	40
6 Conclusioni	42
6.a Risultati ottenuti	43
6.a.a OpenMP (Shared Memory)	43
6.a.b MPI (Distributed Memory)	43

1 Proposta di progetto

Obiettivi

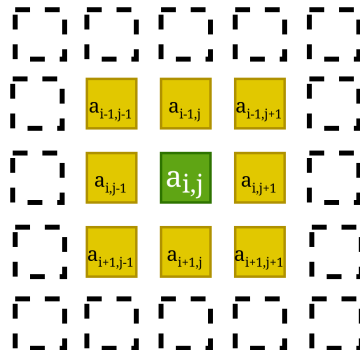


Lo scopo è quello di trasformare una matrice quadrata di numeri reali in una matrice binaria basata sulla media locale di ogni elemento.

- **Matrice di Input (A):** Quadrata di dimensione $N \times N$, con $N \geq 2000$.
- **Matrice di Output (T):** Quadrata di dimensione $N \times N$. Contiene valori interi binari $\{0, 1\}$.

Intorno (I_{ij})

Per ogni elemento $a_{i,j}$ della matrice A , si definisce un **intorno** come la sottomatrice 3×3 centrata nell'elemento stesso:



Obiettivi (ii)



Il valore di ogni elemento $t_{i,j}$ della matrice risultante T viene determinato seguendo questi passaggi:

1. Media dell'intorno ($m_{i,j}$):

- Si calcola la media aritmetica di tutti i 9 elementi appartenenti all'intorno I_{ij} con la formula:

$$m_{i,j} = \frac{1}{9} \sum_{x=i-1}^{i+1} \sum_{y=j-1}^{j+1} a_{x,y}$$

2. Soglia di binaria:

- Se $a_{i,j} > m_{i,j}$ allora $t_{i,j} = 1$
- Se $a_{i,j} \leq m_{i,j}$ allora $t_{i,j} = 0$

3. Complessità:

- Data la dimensione $N \geq 2000$, l'algoritmo deve processare almeno 4×10^6 elementi, rendendo l'ottimizzazione o la parallelizzazione rilevante.

2 Implementazione Seriale

Allocazione delle matrici



Prima di affrontare il problema, è necessario comprendere il modo in cui la matrice viene linearizzata in memoria:

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{pmatrix}$$

Matrice $n \times n$

Memorizzazione Row-Major:

La matrice viene “appiattita” riga dopo riga. Righe adiacenti logicamente sono rappresentate come contigue in memoria.

Rappresentazione in memoria:

$[[a_{11}, \dots, a_{1n}], [a_{21}, \dots, a_{2n}], \dots, [a_{n1}, \dots, a_{nn}]]$

In una matrice $n \times n$, l'elemento in posizione (i, j) può essere acceduto linearmente come: `A[i * N + j]`, e viene allocata nel seguente modo:

```
int *A = malloc(N * N * sizeof(int));  
int *T = malloc(N * N * sizeof(int));
```


Algoritmo



L'algoritmo seriale analizza ogni cella $A(i, j)$, calcola la media dei vicini (inclusa la cella stessa) e assegna un valore binario basato sul confronto tra il valore centrale e la media locale.

- Si cicla lungo le righe e le colonne, definendo la somma dei valori `sum` e il numero di elementi `count`:

```
for (i = 0; i < N; i++) {  
    for (j = 0; j < N; j++) {  
        sum = 0;  
        count = 0;
```

E si definiscono i confini dell'intorno, stabilendo “fino a quanto” si può uscire dalla cella corrente:

```
zmin = (i > 0) ? i - 1 : i;  
zmax = (i < N - 1) ? i + 1 : i;  
wmin = (j > 0) ? j - 1 : j;  
wmax = (j < N - 1) ? j + 1 : j;
```

Algoritmo (ii)



Si cicla successivamente tra i valori dell'intorno:

```
for ( z = zmin; z <= zmax; z++) {  
    for ( w = wmin; w <= wmax; w++) {  
        sum += A[z * N + w];  
        count++;  
    }  
}
```

Infine, si calcola la soglia binaria:

```
T[i * N + j] = (A[i * N + j] * count > sum) ? 1 : 0
```

Nota: È presente una leggera **ottimizzazione**, ovvero l'uso della formula $(A[i][j] * \text{count} > \text{sum})$ al posto della divisione $(\text{sum}/\text{count})$.

3 Implementazione Parallela: MPI

Distribuzione del carico di lavoro



Poiché non è garantito che la dimensione della matrice N sia un multiplo esatto del numero di processi P , in **MPI** occorre definire “artigianalmente” una strategia di partizionamento tra i processi.

- Il nodo **Master** adotta una logica basata sulla divisione intera e sul resto:

base_rows: Ogni processo riceve almeno $N / \text{num_proc}$ righe.

extra_rows: Le righe rimanenti $(N \% \text{num_proc})$ vengono ridistribuite.

- Prima di inviare ai processi le righe di cui si occuperanno (mediante la primitiva **MPI_Scatterv**, poichè sono di dimensione variabile), il nodo **Master** popola due strutture dati che vengono mandate in Broadcast (**MPI_Bcast**) a tutti i processi allocati:

sendcounts: Memorizza il numero totale di elementi assegnati a ogni processo.

senddispls: L'offset iniziale di ciascun processo, calcolato come somma del numero di elementi assegnati ai processi precedenti.

Distribuzione del carico di lavoro (ii)

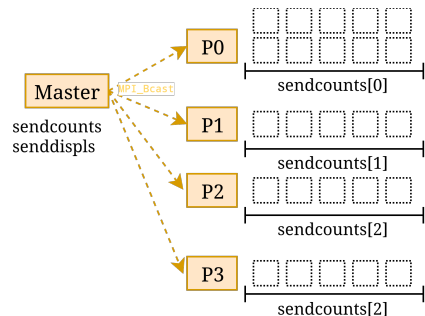


Esempio

Matrice: 5×5 , Numero di processi: 4

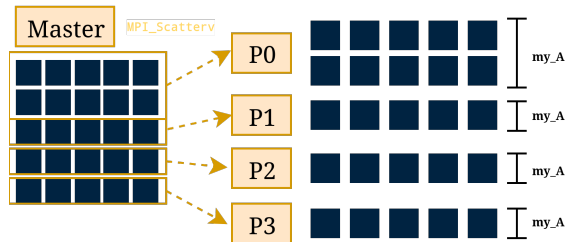
1. MPI_Bcast

Il master comunica a tutti quanto dovranno allocare (`sendcounts`, `senddispls`).



2. MPI_Scatterv

Il master, dopo l'allocazione della matrice, invia i segmenti di dati reali alle memorie locali dei processi.

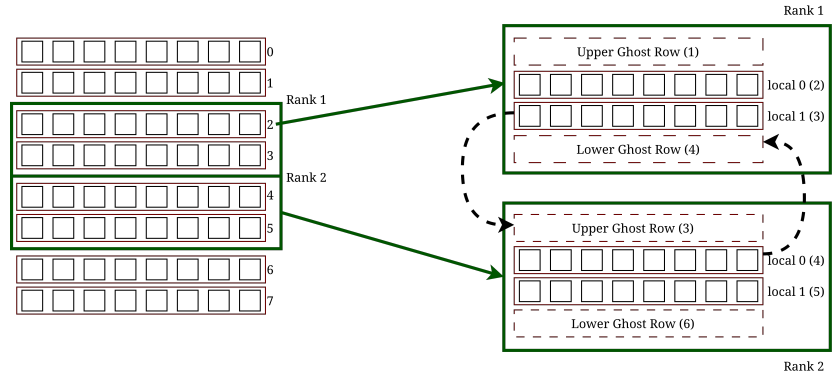


Allocazione e scambio delle Ghost Rows

In un ambiente a **memoria distribuita**, ogni processo opera esclusivamente nel proprio spazio di indirizzamento isolato. Poiché non esiste memoria condivisa, il calcolo dei valori sui bordi richiede uno **scambio** esplicito di messaggi per ottenere i dati residenti sui nodi adiacenti.

Ogni processo alloca quindi localmente la propria porzione di righe (`my_rows = sendcounts[my_rank] / N`) estendendo la struttura dati con due righe supplementari:

- **Upper Ghost Row:** Ospita l'ultima riga del processo precedente (P_{k-1}), se esiste.
- **Lower Ghost Row:** Ospita la prima riga del processo successivo (P_{k+1}), se esiste.



Allocazione e scambio delle Ghost Rows (ii)

88888888888888888888888888888888

Successivamente, ogni processo:

1. Determina chi sono processi adiacenti:

```
int up    = (my_rank > 0)           ? my_rank - 1 : MPI_PROC_NULL;  
int down  = (my_rank < num_proc - 1) ? my_rank + 1 : MPI_PROC_NULL;
```

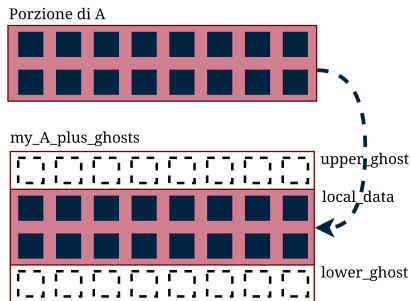
2. Alloca la struttura dati per propria porzione di matrice con due righe extra:

```
int total_rows = my_rows + 2;  
int *my_A_plus_ghosts = calloc(total_rows * N, sizeof(int));
```

3. Definisce un puntatore per ogni sezione della memoria allocata:

```
int *upper_ghost = my_A_plus_ghosts;           // row -1  
int *local_data  = my_A_plus_ghosts + N;       // rows [0 ... my_rows-1]  
int *lower_ghost = my_A_plus_ghosts + (my_rows + 1) * N; // row + my_rows
```

Allocazione e scambio delle Ghost Rows (iii)



4. Il Master distribuisce le porzioni della matrice ai vari processi tramite una `MPI_Scatterv`. Ogni processo riceve `my_rows` righe (pari a `sendcounts[my_rank]` elementi), che vengono mappate direttamente nel buffer `local_data` del processo ricevente:

```
MPI_Scatterv(
    A, sendcounts, senddispls, MPI_INT,
    local_data, sendcounts[my_rank], MPI_INT,
    0, MPI_COMM_WORLD
);
```

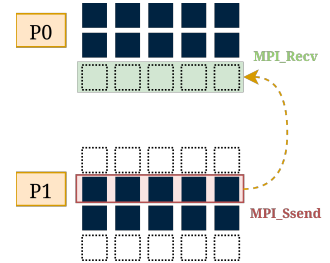
5. Per consentire l'invio e la ricezione delle Ghost Rows, si utilizzano le primitive di comunicazione punto-a-punto. Ci sono diversi approcci che possono essere implementati:
- `MPI_Recv` (bloccante) + `MPI_Ssend`: implementano una comunicazione sincrona di tipo **rendez-vous**.
 - `MPI_Irecv` + `MPI_Isend`: permettono comunicazioni **asincrone**, richiedono `MPI_Wait`.
 - `MPI_Sendrecv`: esegue simultaneamente invio e ricezione, ottimizzando la sincronizzazione.

Allocazione e scambio delle Ghost Rows (iv)

Di seguito è mostrata l'implementazione mediante `Ssend` e `Recv`:

1. Ogni processo (tranne il primo) invia la propria riga di bordo superiore al vicino `up`:

```
if (my_rank > 0 && my_rows > 0) {  
    MPI_Ssend(local_data, N, MPI_INT, up, 100, MPI_COMM_WORLD);  
    MPI_Recv(upper_ghost, N, MPI_INT, up, 200, MPI_COMM_WORLD,  
    MPI_STATUS_IGNORE);  
}
```



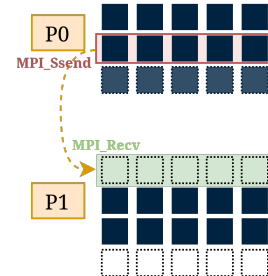
2. Successivamente, si calcola l'indice di memoria dove inizia l'ultima riga locale per l'invio al processo sottostante:

```
int send_down_offset = (my_rows > 0 ? (my_rows - 1) * N : 0);
```

Allocazione e scambio delle Ghost Rows (v)

3. L'invio tramite `Ssend` sincronizza i processi: una volta completata la trasmissione della riga di confine, il processo viene “sbloccato” per la ricezione dal nodo superiore della Ghost Row necessaria:

```
if (my_rank < num_proc - 1 && my_rows > 0) {  
    if (sendcounts[down] > 0) {  
        MPI_Recv(lower_ghost, N, MPI_INT, down, 100,  
MPI_COMM_WORLD, MPI_STATUS_IGNORE);  
        MPI_Ssend(local_data + send_down_offset, N, MPI_INT, down,  
200, MPI_COMM_WORLD);  
    }  
}
```



Calcolo elementi dell'intorno e sogliatura

Dopo lo scambio dei dati, ogni processo itera sulla propria porzione di matrice. Si ricorda che:

Indice (riga logica)	Contenuto
riga 0	Ghost row superiore
righe 1 ... my_rows	Righe di dati locali
riga my_rows + 1	Ghost row inferiore

Per compensare la presenza delle Ghost Rows, l'indice locale deve quindi essere traslato verticalmente:

```
int zmin = (i == 0 && my_rank == 0) ? 0 : -1;           // limite superiore
int zmax = (i == my_rows - 1 && my_rank == num_proc - 1) ? 0 : 1; // limite inferiore
```

E orizzontalmente:

```
int wmin = (j > 0) ? -1 : 0;    // limite destro
int wmax = (j < N - 1) ? 1 : 0; // limite sinistro
```

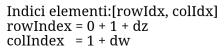
```
for (int dz = zmin; dz <= zmax; dz++) {
    int rowIndex = i + 1 + dz;

    for (int dw = wmin; dw <= wmax; dw++) {
        int colIndex = j + dw;

        sum += my_A_plus_ghosts[rowIndex * N +
                                colIndex];

        count++;
    }
}

my_T[i * N + j] = (local_data[i * N + j] * count > sum)? 1 : 0;
```



Liberazione memoria allocata e raccolta risultati

88888888888888888888888888888888

Si libera la memoria allocata per la matrice locale con le Ghost Rows:

```
free(my_A_plus_ghosts);
```

Il nodo Master invoca la primitiva `MPI_Gatherv` per collezionare i risultati parziali:

```
MPI_Gatherv(  
    my_T, (my_rows * N), MPI_INT,  
    T_raw, sendcounts, senddispls, MPI_INT,  
    0, MPI_COMM_WORLD  
);
```

Ed infine, tutti i nodi liberano la propria porzione di matrice e le strutture dati allocate in precedenza:

```
if (my_rows > 0)  
    { free(my_T); }  
free(sendcounts); free(senddispls);
```

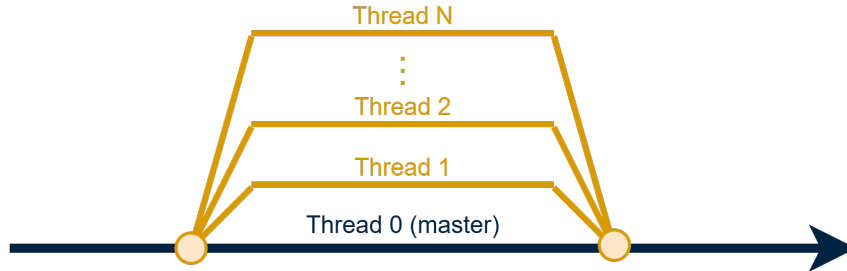
4 Implementazione Parallela: OpenMP

Considerazioni generali

L'implementazione OpenMP è significativamente più semplice a causa della sua gestione dei dati:

- Tutti i thread vedono lo stesso spazio di indirizzamento (**Shared Memory**), ogni thread può leggere direttamente l'intorno di un punto a prescindere dalle righe che gli competono.
- Il compilatore si occupa di parallelizzare il codice mediante clausole singole `#pragma`

Poichè OpenMP segue il modello *Cobegin-Coend*, il Master thread dovrà creare dei worker thread al momento della direttiva `parallel`.



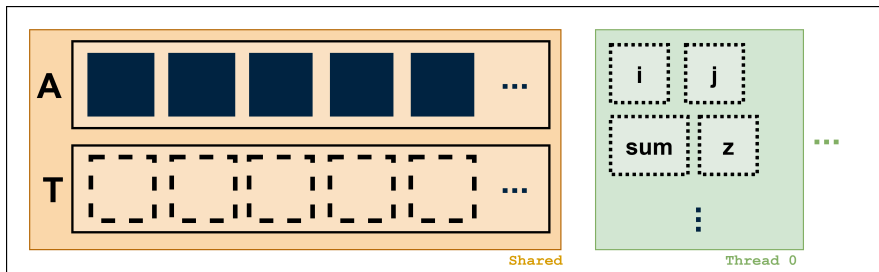
Scelte implementative



Le scelte adottate per l'implementazione dell'algoritmo parallelo hanno riguardato:

- **Visibilità delle Variabili:**

- Matrici A e T : allocate fuori dal ciclo e dichiarate `shared` tra tutti i thread.
- Variabili di supporto al calcolo locale ($i, j, \text{sum}, \text{count}, \text{etc.}$): dichiarate `private` (andranno successivamente inizializzate dai singoli thread).



- **Clausola `for`:** Permette di distribuire le iterazioni di un ciclo tra i thread, e si è scelto di farlo solo per il ciclo “esterno” (che itera sulle righe).

Algoritmo

Si definiscono le clausole OpenMP che permettono di stabilire:

- L'assegnazione statica delle righe nel ciclo **for**;
- Numero di **thread** per processo (come parametro del programma);
- Variabili **condivise**;
- Variabile **private** (non inizializzate).

```
#pragma omp parallel for num_threads(num_threads) schedule(static) \
shared(A, T, N) private(i, j, sum, count, zmin, zmax, wmin, wmax, z, w)
```

L'implementazione dell'algoritmo è la medesima della versione sequenziale:

```
for (i = 0; i < N; i++) {
    for (j = 0; j < N; j++) {
        sum = 0;
        count = 0;

        // ...
    }
}
```

5 Benchmark

Setup sperimentale

L'obiettivo è quello di valutare la scalabilità dell'algoritmo al variare delle risorse di calcolo e della dimensione del problema. I tempi riportati rappresentano la **media** e la relativa **deviazione standard** di 3 misurazioni indipendenti per ogni configurazione.

Le due analisi effettuate riguardano:

1. **Strong Scaling**: Dimensione del problema fissa ($N = 10.000$), unità di calcolo parallelo crescente ($P = 1...48$).
2. **Weak Scaling**: Dimensione del problema ($N = 5000\sqrt{P}$) e unità di calcolo parallelo ($P = 1...48$) crescenti.

Le metriche di valutazione utilizzate sono:

1. **Speedup (S)**: Rapporto tra il tempo in sequenziale e il tempo in parallelo.
 - Per lo **Strong Scaling**: $S(P) = \frac{T(1)}{T(P)}$
 - Per il **Weak Scaling** (Speedup Scalato): $S(P) = P \cdot \left(\frac{T(1)}{T(P)} \right)$
2. **Efficiency (E)**: Rapporto tra speedup e unità di calcolo parallelo utilizzate: $E(P) = \frac{S(P)}{P}$

L'analisi sperimentale ha considerato tre configurazioni: **MPI intra-nodo** (più processi MPI indipendenti su un singolo nodo), **MPI inter-nodo** (un processo MPI per nodo, con aumento progressivo del numero di nodi utilizzati) e **OpenMP** inter-nodo.

Setup sperimentale (ii)



Le direttive **Slurm** utilizzate sulla macchina *Galileo-100* del *Cineca* sono le seguenti:

Configurazione	Nodes	Tasks	CPU per Task	N
MPI , Strong scaling intra-nodo	1	1...48	1	10.000
MPI , Weak scaling intra-nodo	1	1...48	1	$5000\sqrt{P}$
MPI , Strong scaling inter-nodo	1...24	1 per nodo	1	10.000
MPI , Weak scaling inter-nodo	1...24	1 per nodo	1	$5000\sqrt{P}$
OpenMP , Strong scaling	1	1	48 (1...48 thread)	10.000
OpenMP , Weak scaling	1	1	48 (1...48 thread)	$5000\sqrt{P}$

Nota: Il numero di nodi indipendenti allocati sono limitati a 24 a causa di limiti del cluster.

MPI (ii)

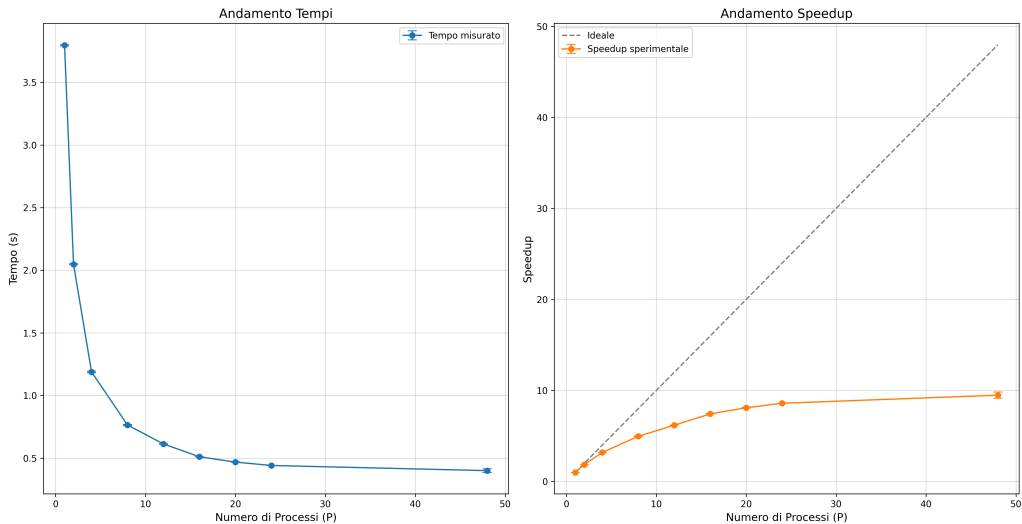


Figure 1: Strong Scaling inter-nodo - MPI

MPI (iii)



5.b.b Weak Scaling intra-nodo

- Nel caso di **weak scaling** si osserva un comportamento analogo.

P	N	Tempo medio	Dev. standard	Speedup	Efficiency
1	5000	0.9482	0.0016	1	1
2	7071	1.0245	0.0016	1.851	0.9255
4	10000	1.1913	0.0008	3.1836	0.7959
8	14142	1.5363	0.0017	4.9373	0.6172
12	17320	1.871	0.0041	6.0813	0.5068
16	20000	1.9983	0.0054	7.5919	0.4745
20	22360	2.2752	0.0099	8.3348	0.4167
24	24494	2.564	0.0068	8.8752	0.3698
48	34641	4.6097	0.0085	9.8732	0.2057

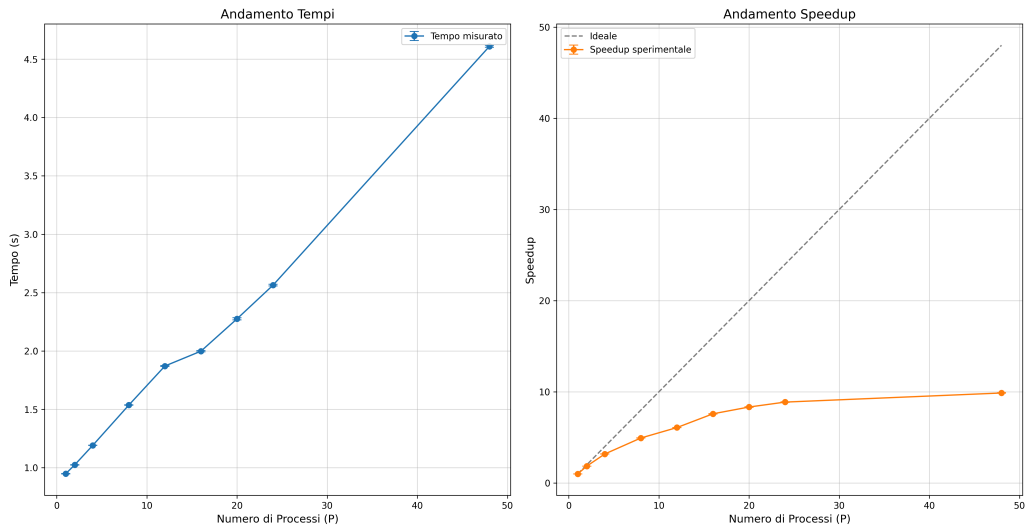
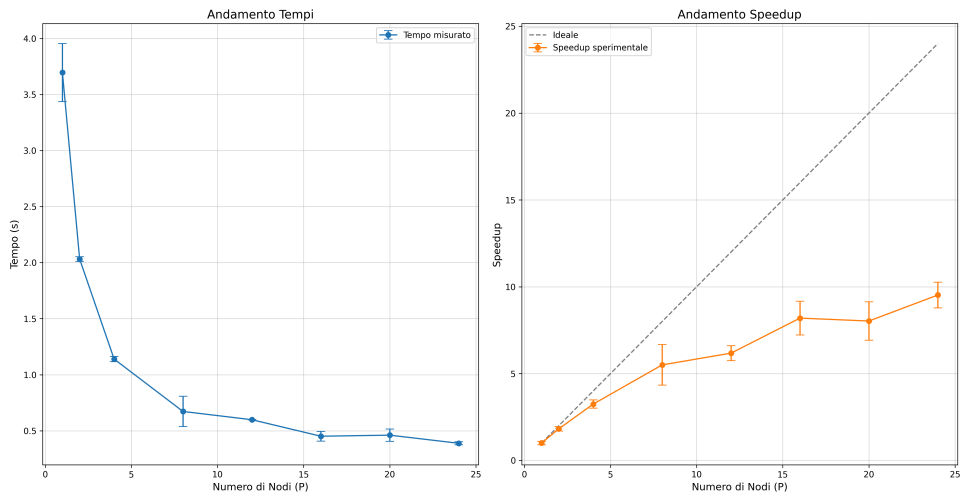


Figure 2: Weak Scaling inter-nodo - MPI

5.b.c Strong Scaling inter-nodo

- La similitudine tra il comportamento MPI **intra-nodo** e **inter-nodo** conferma il predominio dell'overhead di comunicazione rispetto al lavoro computazionale, che limita lo scaling in entrambe le configurazioni.

P	N	Tempo medio	Dev. standard	Speedup	Efficiency
1	10000	3.6967	0.2585	1	1
2	10000	2.032	0.0207	1.8192	0.9096
4	10000	1.1396	0.0236	3.2438	0.811
8	10000	0.6721	0.1351	5.5006	0.6876
12	10000	0.5984	0.0011	6.1781	0.5148
16	10000	0.4511	0.0432	8.1946	0.5122
20	10000	0.4605	0.0543	8.0272	0.4014
24	10000	0.3882	0.0135	9.5235	0.3968



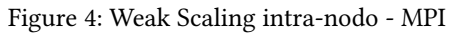
MPI (vii)



5.b.d Weak Scaling inter-nodo

- Si osserva anche qui un comportamento analogo al caso intra-nodo, con benefici di scaling limitati.

P	N	Tempo medio	Dev. standard	Speedup	Efficiency
1	5000	0.9623	0.0235	1	1
2	7071	1.0243	0.0131	1.879	0.9395
4	10000	1.1515	0.0093	3.3429	0.8357
8	14142	1.4222	0.0342	5.4132	0.6767
16	20000	1.949	0.0812	7.9001	0.4938
20	22360	2.7806	0.0055	6.9219	0.3461
24	24494	2.7406	0.0685	8.4274	0.3511



5.c.a Strong Scaling

- OpenMP mantiene un'efficienza straordinaria, sempre superiore al **93%**. L'assenza di comunicazioni esplicite fa sì che i thread accedano **direttamente** alla memoria condivisa.

P	N	Tempo medio	Dev. standard	Speedup	Efficiency
1	10000	3.1012	0.0201	1	1
2	10000	1.5538	0.0024	1.9959	0.9979
4	10000	0.7778	0.0023	3.9874	0.9968
8	10000	0.3916	0.0021	7.9198	0.99
12	10000	0.2592	0.0017	11.9646	0.997
16	10000	0.1975	0.004	15.7052	0.9816
20	10000	0.1564	0.0007	19.824	0.9912
24	10000	0.1321	0.0018	23.4784	0.9783
48	10000	0.0689	0.0005	44.9831	0.9371

OpenMP (ii)

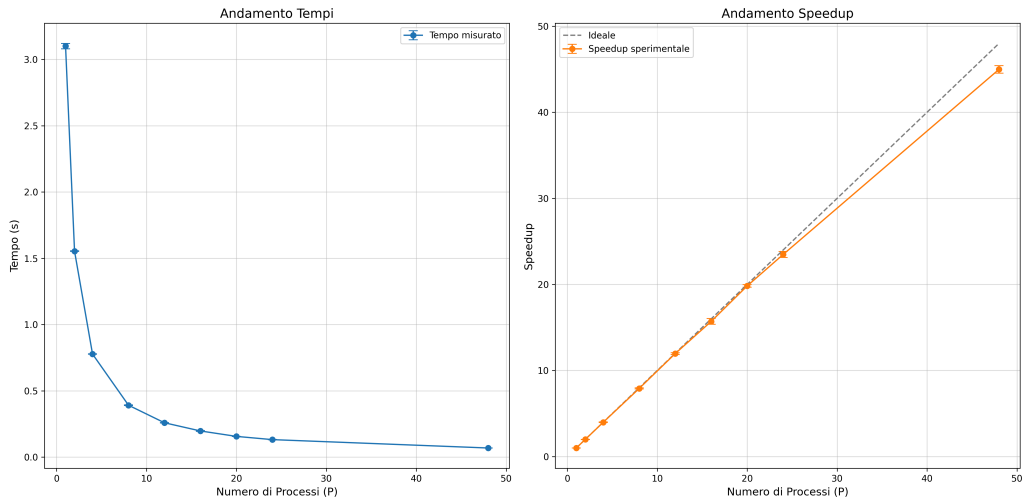


Figure 5: Strong Scaling - OpenMP

OpenMP (iii)



5.c.b Weak Scaling

- Il tempo di esecuzione rimane pressoché costante ($\approx 0.77s$), poichè ogni core riceve la stessa **quantità** di lavoro computando in parallelo **senza conflitti**, e processando matrici più grandi nello stesso tempo di quelle piccole.

P	N	Tempo medio	Dev. standard	Speedup	Efficiency
1	5000	0.7732	0.0036	1	1
2	7071	0.7737	0.0045	1.9988	0.9994
4	10000	0.7854	0.0152	3.938	0.9845
8	14142	0.7763	0.0035	7.9684	0.9761
12	17320	0.7732	0.0004	12.0009	0.9971
16	20000	0.7826	0.0084	15.8075	0.988
20	22360	0.7816	0.0047	19.7861	0.9893
24	24494	0.7897	0.007	23.4992	0.9791
48	34641	0.8099	0.0009	45.8262	0.9547

OpenMP (iv)

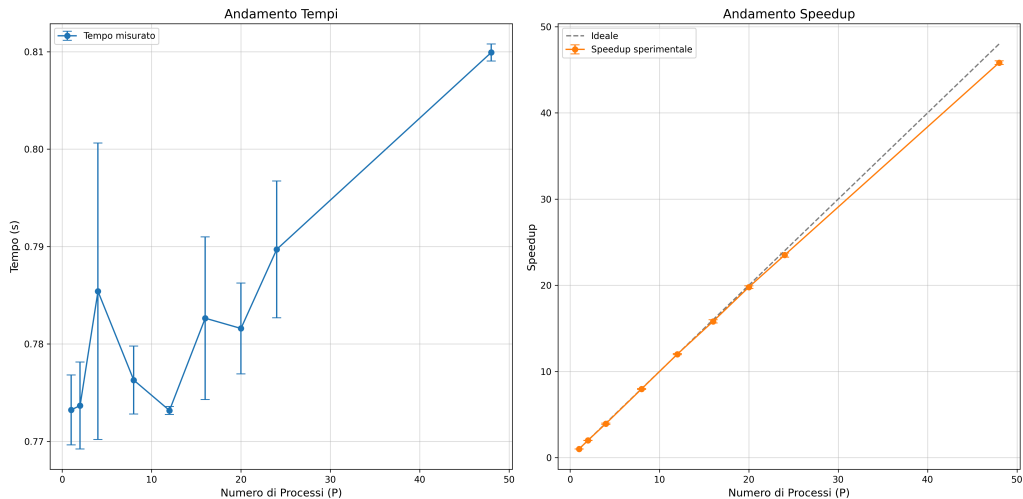


Figure 6: Weak Scaling - OpenMP

6 Conclusioni

6.a.a OpenMP (Shared Memory)

- **Speedup:** Quasi ideale (Efficienza sempre $> 95\%$).
- **Punti di forza:** Accesso diretto ai dati, zero overhead di comunicazione.
- **Criticità:** Vincolato alle risorse di un singolo nodo.

6.a.b MPI (Distributed Memory)

- **Speedup:** Limitato (Efficienza anche $< 20\%$).
- **Punti di forza:** Permette di eseguire su cluster multi-nodo per N estremamente grandi.
- **Criticità:** Performance limitate dall'overhead di comunicazione.

In base ai calcoli effettuati, **OpenMP** risulta la soluzione più efficiente per la natura del problema. **MPI** è necessario solo quando la memoria del singolo nodo non è più sufficiente.

Questa slide è lasciata intenzionalmente vuota.